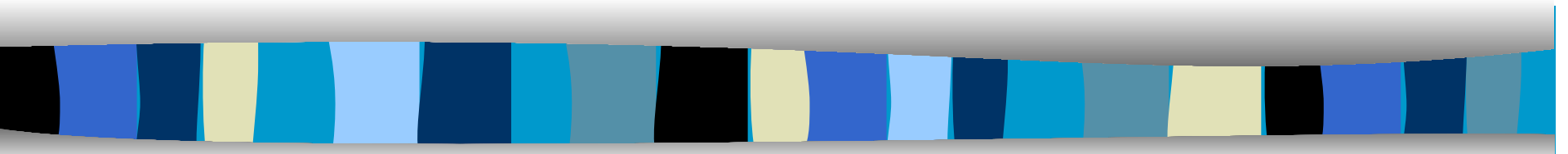


Object Representation

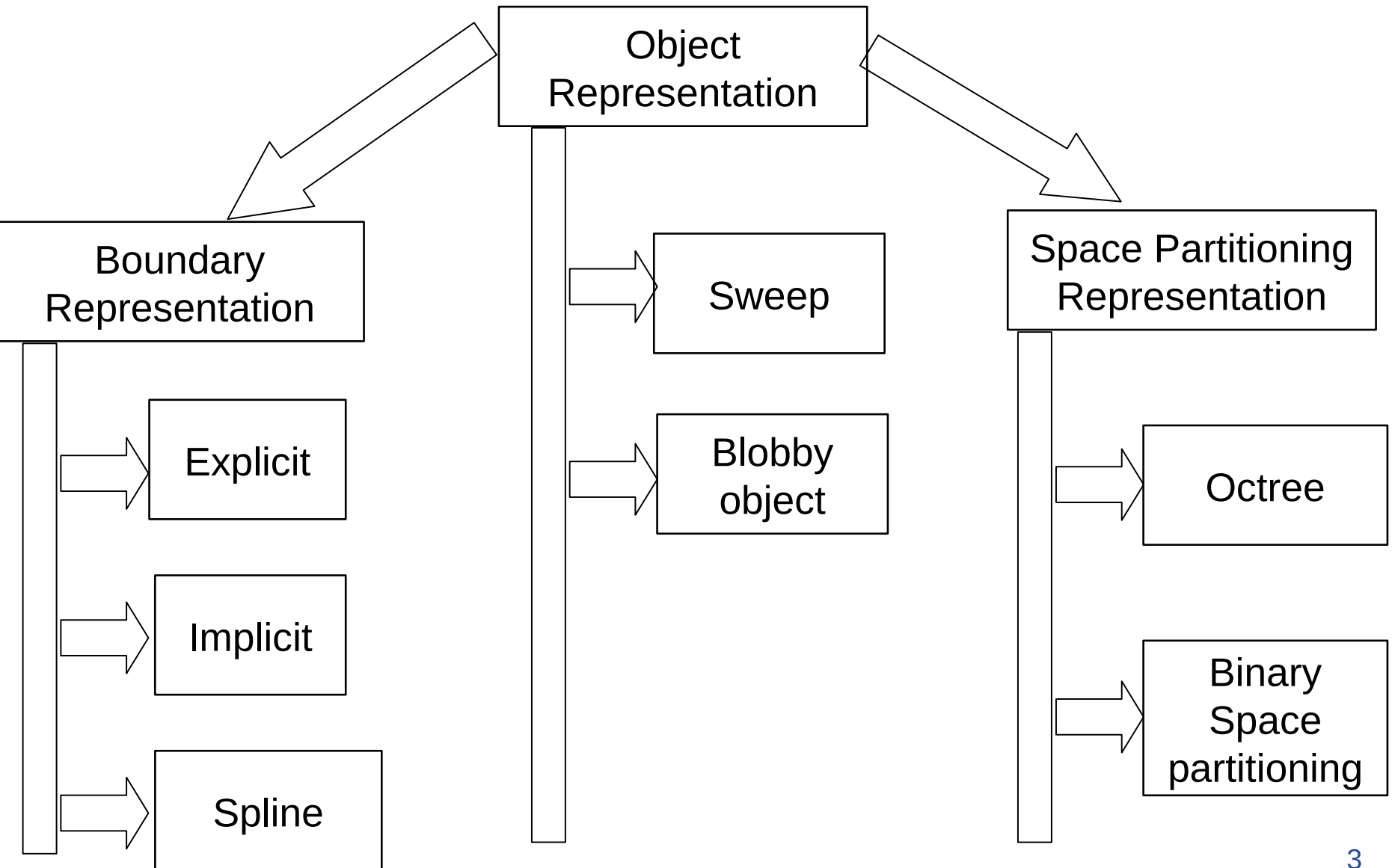


Polygon Table, Mesh Table, Natural
Cubic Spline, Hermite Spline, Bezier
Curve, B-spline

Object Representation

- A scene synthesized with computer can contain objects with different shape and sizes.
- Objects can be very simple to complex.
- We need to represent object in appropriate way so that realistic scene can be rendered.
- Creating realistic effects involves lots of computations.
- We represent objects with edges, vertices for sampled points at different location.

Object Representation



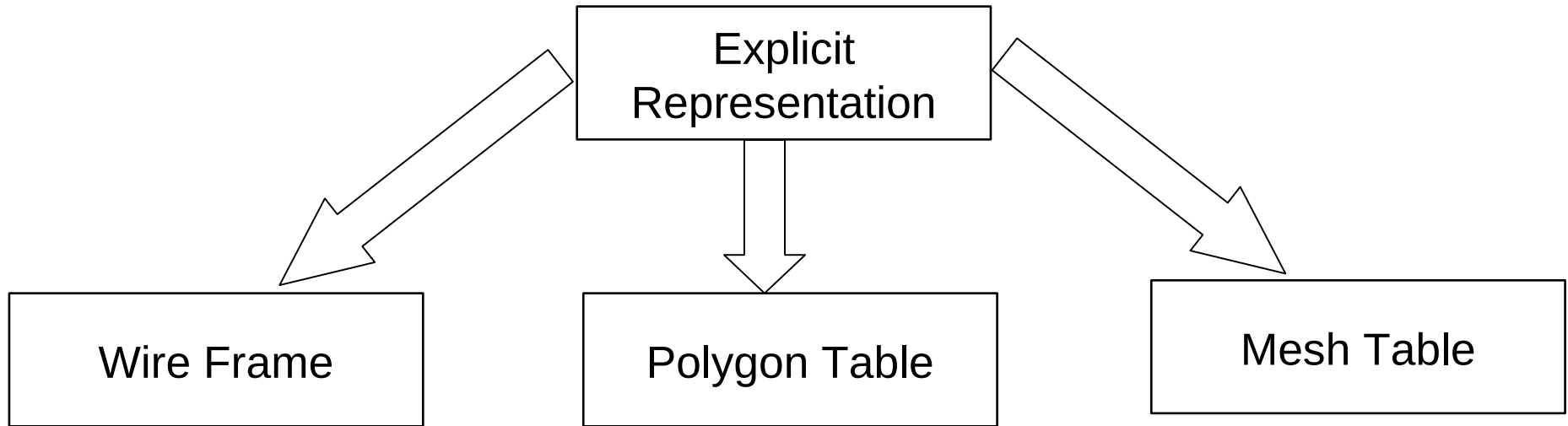
Boundary Representation

- In boundary representation, objects are represented by representing its boundary lines and surfaces.
- The surfaces can be polygonal or curves.
- For e.g. a cube has six faces, and 8 vertices.
- So, we represent cube by these 6 boundary faces.
- Boundary representation can be
- **Explicit**
- **Implicit**
- **Spline**

Explicit Representation

- In this method, objects are represented by specifying vertices, edges, surfaces explicitly.
- It is very simplest method.
- However, if the object is complex, then it cannot represent it correctly.

Explicit Representation



Wire Frame Model

- Perhaps oldest way of representing solids.
- Any solid is represented with edges and vertices.
- Consists of two tables, vertex table and edge table.
- Each entry of vertex table records a vertex and its coordinate values.
- Each entry of edge table records starting vertex and ending vertex of an edge.
- It does not have face information.

Wire Frame Model

- It is simplest model but ambiguous.
- We cannot tell the direction of the opening
- Cube can look like it has hole.
- However it is efficient.
- It can be used for rendering preview of an object.

Wire Frame Model

Vertex Table			
Vertex #	x	y	z
1	1	1	1
2	1	-1	1
3	-1	-1	1
4	-1	1	1
5	1	1	-1
6	1	-1	-1
7	-1	-1	-1
8	-1	1	-1

(a)

Edge Table		
Edge #	Start Vertex	End Vertex
1	1	2
2	2	3
3	3	4
4	4	1
5	5	6
6	6	7
7	7	8
8	8	5
9	1	5
10	2	6
11	3	7
12	4	8

(b)

Wire Frame Model

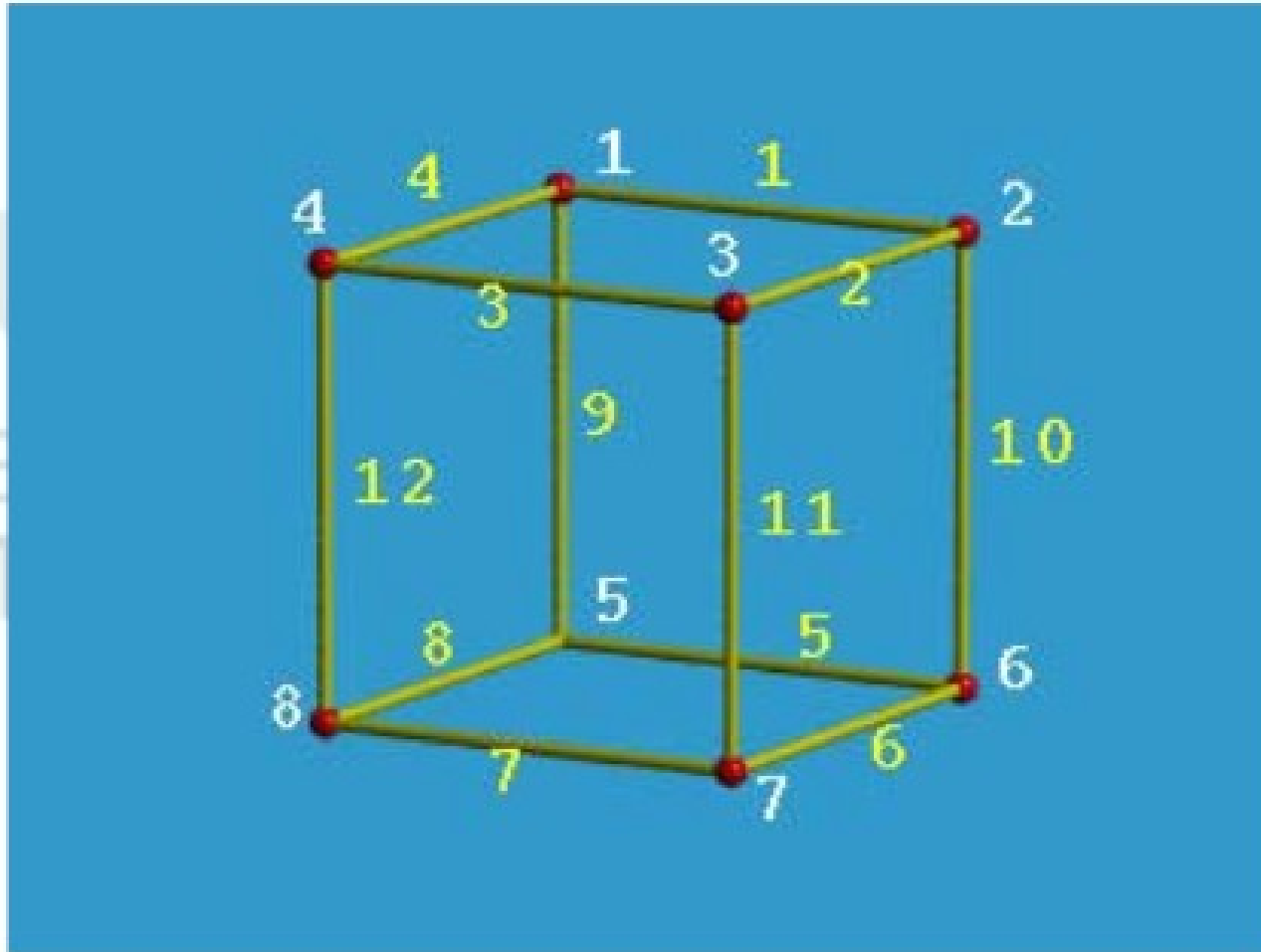


Figure 6.1 : Wire-frame Model of a Unit Cube

Polygon Table

- Polygon Table represents the object with set of polygons.
- It uses one polygon for each planner face of the object.
- It is the most commonly used for representing 3D graphics.
- It is simplest and speeds up the scene rendering process.
- Each polygon can be represented with linear equations.

Polygon Table

- To represent the polygons of the object, it uses
- Geometric Table
- Attribute Table
- **Attribute table** includes information about surface such as degree of transparency, reflectivity, and texture.
- **Geometric Table** consists of vertex, edge and surface information.

Polygon Table

Geometric tables consists of:

- VERTEX TABLE
- EDGE TABLE
- POLYGON SURFACE TABLE

	VERTEX TABLE
V1	X1,Y1,Z1
V2	X2,Y2,Z2
V3	X3,Y3,Z3
V4	X4,Y4,Z4
V5	X5,Y5,Z5

	EDGE TABLE
E1	V1,V2
E2	V2,V3
E3	V3,V1
E4	V3,V4
E5	V4,V5
E6	V5,V1

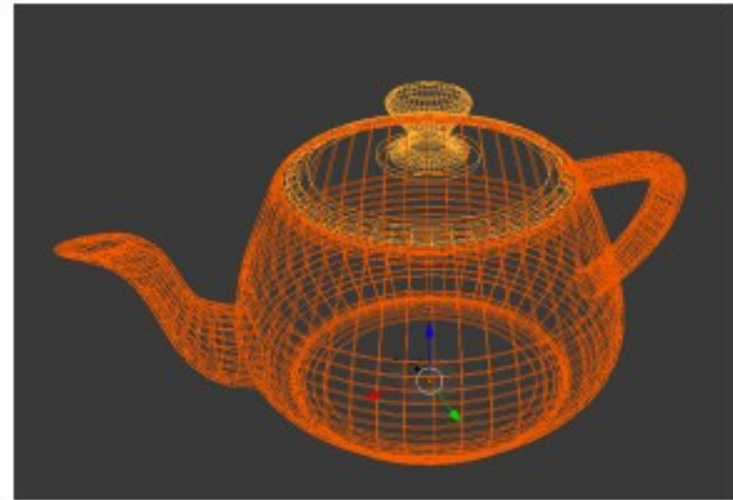
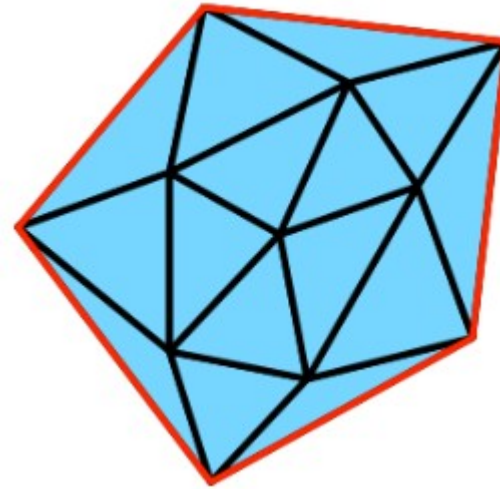
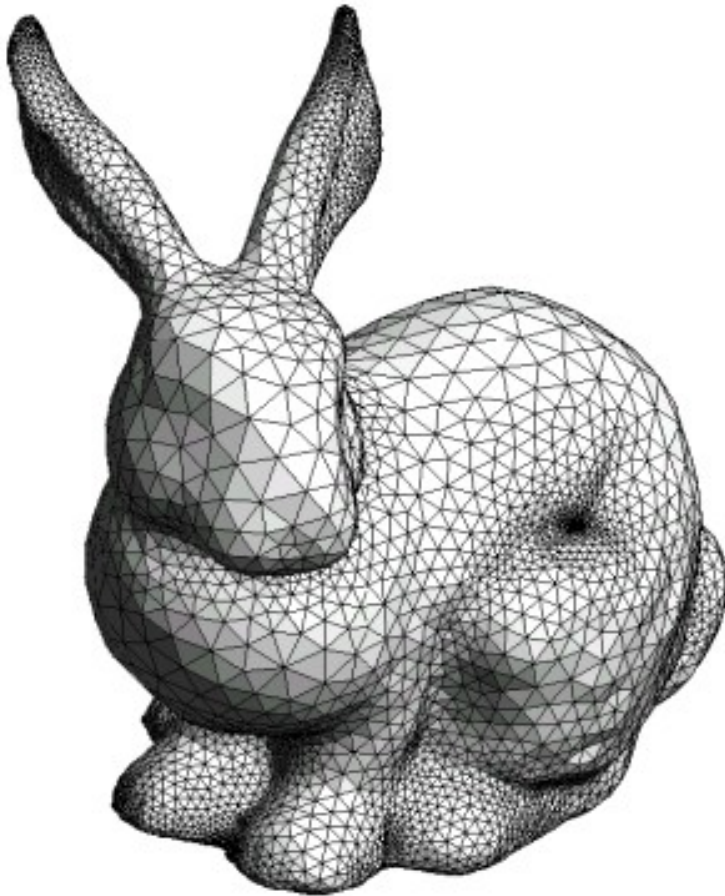
POLYGON-SURFACE TABLE
S1:E1,E2,E3
S2:E3,E4,E5,E6

Mesh Table

- One problem with polygon table is that it can represent objects with planner surface only.
- It cannot represent objects with curved surface.
- To represent curved surface we can use mesh table.
- In mesh table, the curved surface is divided into small planner surface.
- These planner surface now can be represented with triangle or other polygons.
- So, mesh table also consists of vertex, edges, face tables.

Mesh Table

Mesh structures



Objects represented as a set of polygons

Mesh Table

- **Advantages:**

- It can be used to model almost any object.
- Its easy to transform and draw on computer screen.

- **Disadvantages:**

- Not convenient as it is very difficult to determine how many polygons should be used.
- It only approximate the object.
- Cumbersome to represent large vertices, edges and faces.
- Difficult to simulate objects such as hair or liquid

Implicit Representation

- Implicit method represents surfaces in terms of function form
- Some standard objects have standard equations.
- Equations can be parametric or non-parametric.

Implicit Representation

1. Sphere:

For the set of surface points (x, y, z) the spherical surface is represented as: $x^2+y^2+z^2 = r^2$, with radius r and centered at co-ordinate origin.

2. Ellipsoid:

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{c^2} = 1$$

where (x, y, z) is the surface points and a, b, c are the radii on X, Y and Z directions respectively.

Implicit Representation

3. Elliptic Paraboloid

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = z$$

4. Hyperbolic Paraboloid

$$\frac{x^2}{a^2} - \frac{y^2}{b^2} = z$$

5. Elliptic Cone

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} - \frac{z^2}{c^2} = 0$$

Implicit Representation

Example: A Circle of radius R

- Implicit:

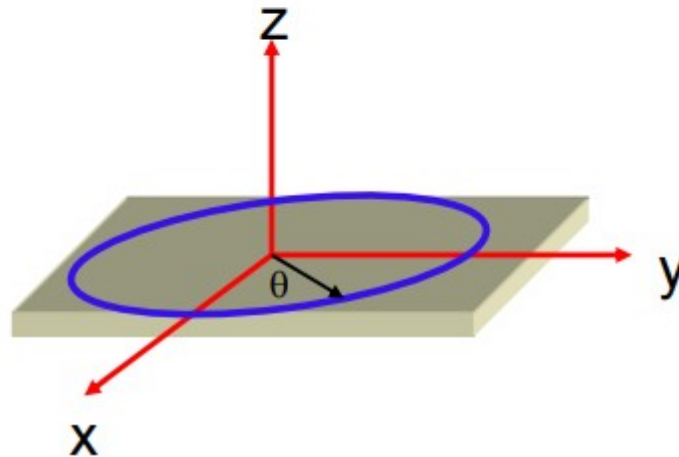
$$x^2 + y^2 + z^2 - R^2 = 0 \quad \& \quad z = 0$$

- Parametric:

$$x(\theta) = R \cos(\theta)$$

$$y(\theta) = R \sin(\theta)$$

$$z(\theta) = 0$$



Implicit Representation

- Any plane can be represented as
- **$Ax+By+Cz+D=0$**

Implicit Representation

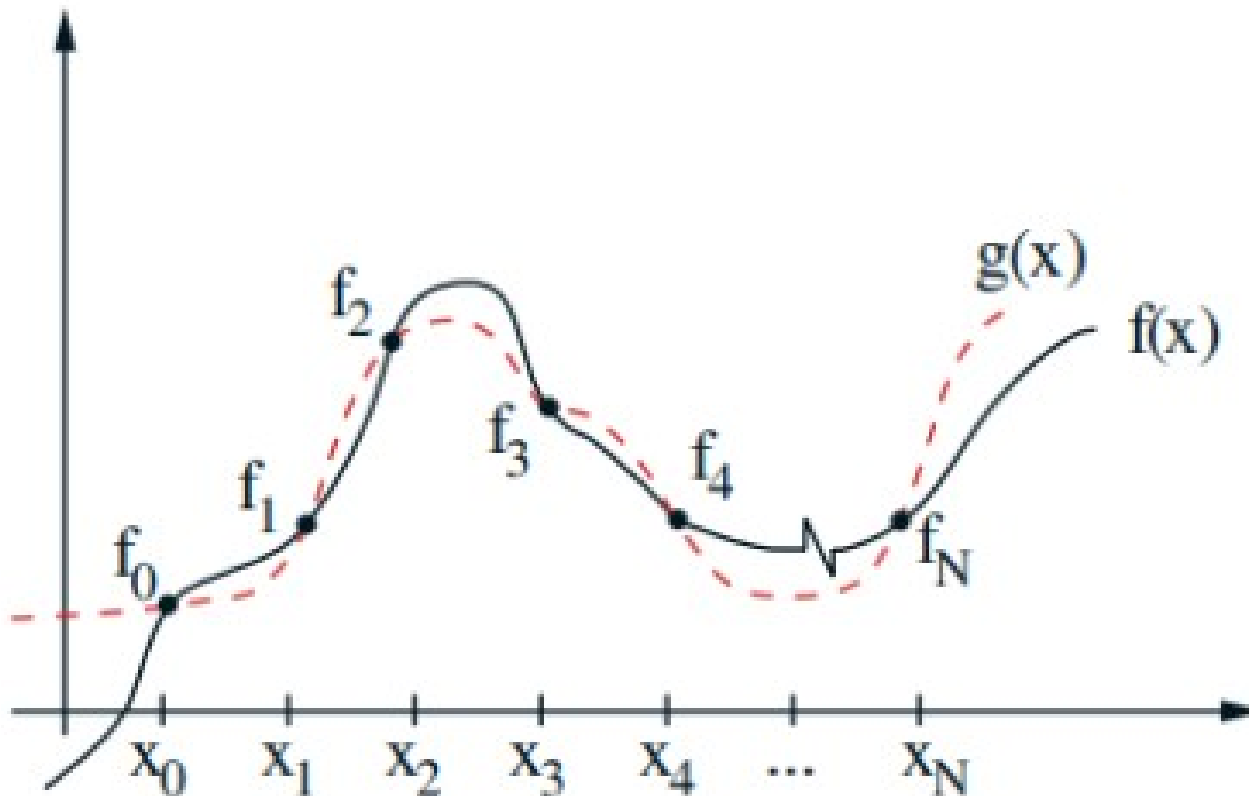
- **Lagrange Interpolation**
- We have represented some standard objects with some standard equations.
- However all the objects do have readymade equations.
- So, we have to find equations of objects or the curves that make up the object.
- One method of finding the equation of curves is lagrange interpolation.

Implicit Representation

- **Lagrange Interpolation**
- Let us consider a polynomial $g(x)$ of degree n
- $g(x)=a_0+a_1x+a_2x^2+..... +a_nx^n$
- Here, $a_0, a_1, a_2 \dots a_n$ coefficients.
- If we get values of these coefficients, then we will have the equation of curve.
- Let $g(x)$ passes through $n+1$ points of the curve that represents the object.

Implicit Representation

- **Lagrange Interpolation**



Implicit Representation

- **Lagrange Interpolation**
- Solving all these $n+1$ curves, we will get the values of coefficients.
- This gives the equation of the n degree curve

$$g(x_o) = f_o \quad \Rightarrow \quad a_o + a_1x_o + a_2x_o^2 + \dots + a_Nx_o^N = f_o$$

$$g(x_1) = f_1 \quad \Rightarrow \quad a_o + a_1x_1 + a_2x_1^2 + \dots + a_Nx_1^N = f_1$$

:

:

$$g(x_N) = f_N \quad \Rightarrow \quad a_o + a_1x_N + a_2x_N^2 + \dots + a_Nx_N^N = f_N$$

Implicit Representation

- **Lagrange Interpolation**
- X: 0 1 3
- F: 1 3 2
- Let $g(x)=a_0+a_1x+a_2x^2$ be the curve.
- Now, $g(0)=a_0+0+0=1$ or, $a_0=1$
- $g(1)=a_0+a_1+a_2=3$ or, $a_1=2-a_2$
- $g(3)=a_0+3a_1+9a_2=2$
- or, $1+3(2-a_2)+9a_2=2$
- Or, $6-3a_2+9a_2=1$
- Or, $6a_2=-5$
- Or, $a_2=-5/6$

Implicit Representation

- **Lagrange Interpolation**
- X: 0 1 3
- F: 1 3 2
- $a_1=2-a_2=2+5/6=17/6$
- So, so interpolated curve is
- $g(x)=1+17/6x-5/6x^2$

Problems Implicit Representation

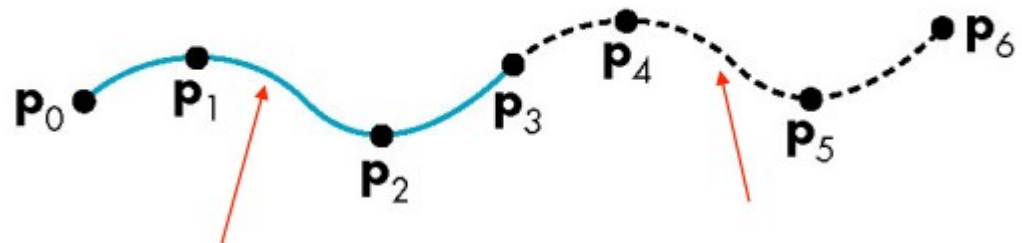
- We can get the equation of curves using lagrange interpolation method.
- However, this is not efficient for representing complex objects.
- To represent complex surface, we need to interpolate large number of points.
- In this process we need to solve $n+1$ equation of n degree.
- Where n can be very large say 100000.
- This is very complex.
- The better solution in spline representation.

Spline Representation

- In this method, the large points of a complex objects are grouped into small set of points.
- The points in each group is called **control points**.
- From each group a small degree curve is interpolated.
- All curves are combined to give large single curve called **spline**.
- For representing small curves, we can use parametric equations.
- Parametric equations are flexible and easy to operate.

Spline Representation

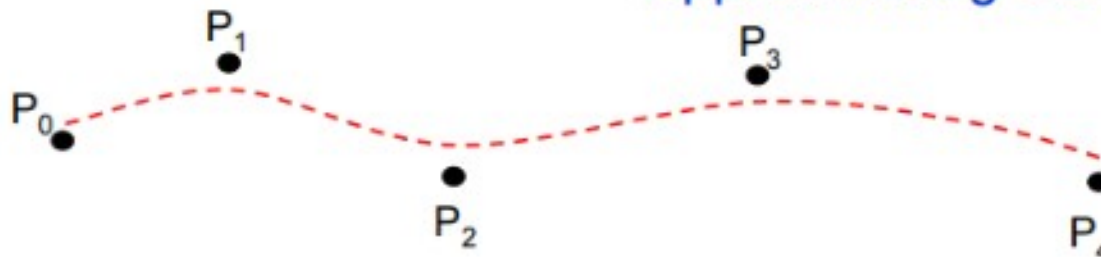
- P_0, p_1, p_2, p_3 control points of first curve.
- P_3, p_4, p_5, p_6 control points of second curve



Interpolating Curve



Approximating Curve



Spline Representation

- The parametric equation is function of parameter t .
- The parametric equation of degree 1 is
- $f(t)=a +bt$.
- The quadratic parametric equation is
- $f(t)=a+bt+ct^2$.
- The cubic parametric equation is
- $f(t)=a+bt+ct^2+dt^3$
- Here, a, b, c, d are coefficients.
- These equations must be splitted into x and y form to compute x and y co-ordinate.

Spline Representation

- The x-coordinate of the curve $f(t)=a+bt$ is
- $x(t)=f^x(t)$
- $y(t)=f^y(t)$
- We can represent any polynomial in parametric form using its basis matrix.
- Each polynomial has its fixed **basis matrix**.
- It will be easy to present any polynomial using basis matrix in parametric form.

Spline Representation

- Let us compute the **basis matrix** for polynomial of **degree 1**.

Spline Representation

- **Basis matrix** for polynomial of **degree 1**.

$$f(t) = a + bt,$$

Where

$f(t)$ is function of parameter t .

Range of parameter t is $0 \leq t \leq 1$

Now, $f(t) = U.A$,

where $U = \begin{bmatrix} 1 & t \end{bmatrix}$ and

$$A = \begin{bmatrix} a \\ b \end{bmatrix}$$

This shows that if we know A ,

then we can represent polynomial of degree 1.

Spline Representation

- **Basis matrix** for polynomial of **degree 1**.

We can get control points p_0 and p_1 by putting $t = 0$ and $t = 1$ respectively.

$$p_0 = f(t = 0) = a + b \cdot 0$$

$$p_1 = f(t = 1) = a + b \cdot 1$$

These equations can be represented as
 $P = C \cdot A$,

Where P is either $P = \begin{bmatrix} p_0^x \\ p_1^x \end{bmatrix}$ or $P = \begin{bmatrix} p_0^y \\ p_1^y \end{bmatrix}$

$$\text{and } C = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \quad A = \begin{bmatrix} a \\ b \end{bmatrix}$$

Spline Representation

- **Basis matrix** for polynomial of **degree 1**.

Now, We have

$$A = C^{-1}.P$$

Substitute A in $f(t)$ we get,

$$f(t) = U.C^{-1}.P = U.B_{linear}.P,$$

Where, $B_{linear} = C^{-1}$ is basis matrix of polynomial degree 1.

So, we can get any point on the line

by just changing the value of t in U matrix.

Spline Representation

- **Basis matrix** for polynomial of **degree 1**.
- Here, $B_{linear} = C^{-1} = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$ is called Basis Matrix
- It is unique for each degree of polynomial.
- So, we can represent specific degree polynomial using fixed basis matrix.
- We just need to evaluate Basis Matrix.
- We can combine these pieces that have same degree and same constants to represent splines.

Spline Representation

- **Basis matrix** for polynomial of **degree 1**.
- let us consider a line joining points (2,3) and (4,6)
- Slope, $m = \frac{6-3}{4-2} = \frac{3}{2}$
- Intercept, $c = y - mx = 3 - \frac{3}{2} \cdot 2 = 0$
- equation of line is, $y = \frac{3}{2}x$
- we know that points at $t=0$ is (2,3) and at $t=1$ is (4,6)
- let us find other points at $t=0.25$ and $t=0.5$ using basis matrix B_{linear}
$$\begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

Spline Representation

- **Basis matrix** for polynomial of **degree 1**.
- We have $p^x = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$, and $p^y = \begin{bmatrix} 3 \\ 6 \end{bmatrix}$
- At $t=0.25$, $U = \begin{bmatrix} 1 & 0.25 \end{bmatrix}$ and
- Now, $f^x(t = 0.25) = UB_{linear}p^x = \begin{bmatrix} 1 & 0.25 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 4 \end{bmatrix} = 2.5$
- Similarly, $f^y(t = 0.25) = U \cdot B_{linear} \cdot p^y = 3.75$
- We can see that $(2.5, 3.75)$ satisfies $y = \frac{3}{2} \cdot x$
- Similarly, we can calculate point at $t = 0.5$

Spline Representation

- **Basis matrix** for polynomial of **degree 1**.

$$f(t) = a + bt$$

Where, we choose parameter t such as $0 \leq t \leq 1$

Now,

$$x(t) = a_x + b_x t \text{ and}$$

$$y(t) = a_y + b_y t$$

So,

$$p_0^x = f^x(t = 0) = a_x + b_x \cdot 0 \text{ and}$$

$$p_0^y = f^y(t = 0) = a_y + b_y \cdot 0$$

Similarly,

$$\text{So, } p_1^x = f^x(t = 1) = a_x + b_x \cdot 1 \text{ and}$$

$$p_1^y = f^y(t = 1) = a_y + b_y \cdot 1$$

Spline Representation

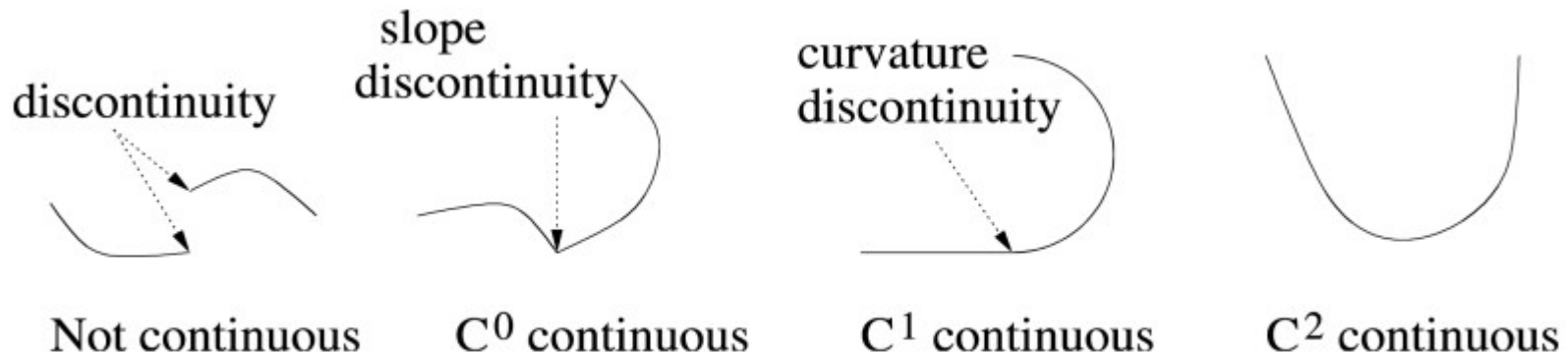
- **Parametric Continuity:**
- When we construct the spline from different curves, the continuity of spline must be considered
- Continuity can be
- Order 0 or $C0$
- Order 1 or $C1$
- Order 2 or $C2$.

Spline Representation

- **Parametric Continuity:**
- In **C0 continuity**, $f(t_1)=f(t_2)$ where t_1 is ending parameter of first curve and t_2 is starting parameter of second curve.
- It means two curve meets.
- In **C1 continuity** the first order parametric derivative of adjoining curves at common boundary are equal.
- In **C2 continuity** both first and second order derivatives are equal.

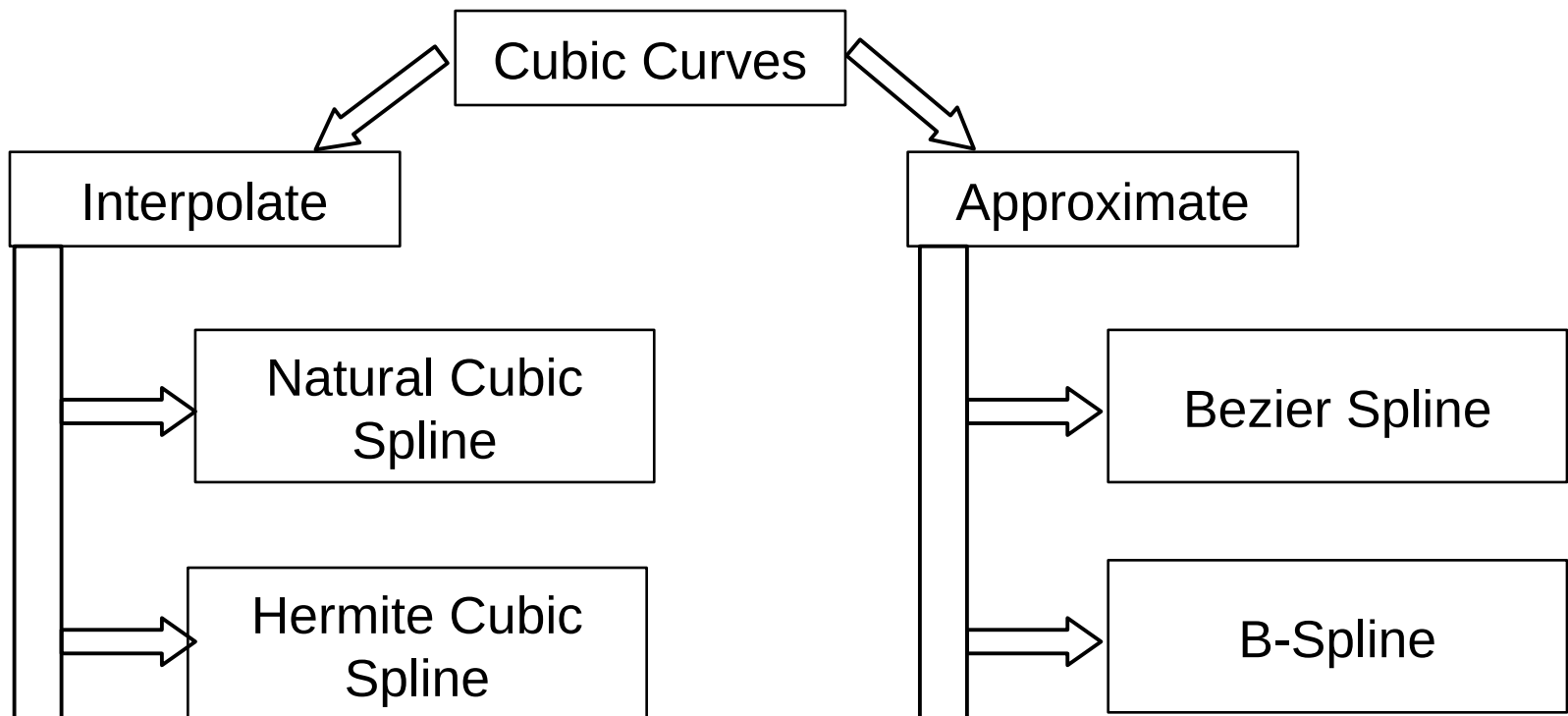
Spline Representation

- **Parametric Continuity**



Spline Representation

- **Cubic Curves:**
- The best way to present spline is joining different cubic curves.
- There are different types of cubic curves.



Natural Cubic Spline

- The first spline used in computer graphics is natural cubic spline.
- It is made up of pieces of 3rd degree polynomials.
- Each piece is interpolated with four control points p_0 , p_1 , p_2 , and p_3 .
- p_0 and p_3 are boundary points at time $t=0$ and $t=1$
- p_1 and p_2 are first and second derivative of polynomial at $t=0$.

Natural Cubic Spline

$$f(t) = a + bt + ct^2 + dt^3$$

$$f'(t) = b + 2ct + 3dt^2$$

$$f''(t) = 2c + 6dt$$

$$p_0 = f(t = 0) = a + b.0 + c.0 + d.0$$

$$p_1 = f'(t = 0) = 0 + b + 2c.0 + 3d.0$$

$$p_2 = f''(t = 0) = 0 + 0 + 2c + 6d.0$$

$$p_3 = f(t = 1) = a + b + c + d$$

From above system of linear equations, we get

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}, \text{ So, we have } C^{-1} = B_{\text{natural}} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0.5 & 0 \\ -1 & -1 & -0.5 & 1 \end{bmatrix}$$

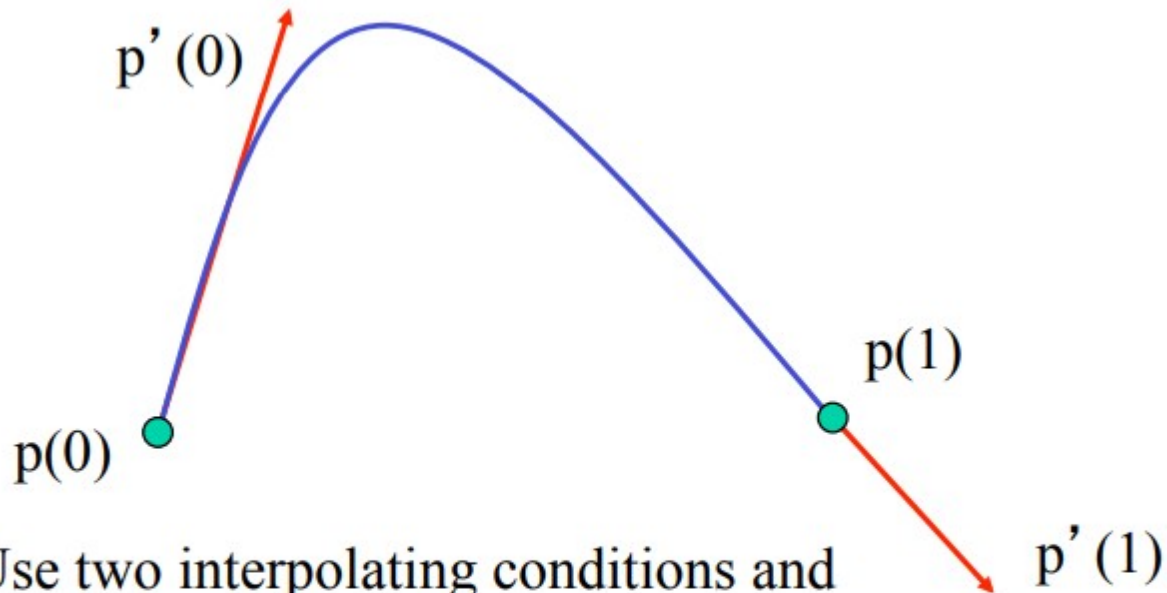
Natural Cubic Spline

- Natural cubic spline is represented with $\mathbf{f(t)} = \mathbf{U} \mathbf{B}_{\text{natural}} \mathbf{P}$
- Where $\mathbf{U} = [1 \ t \ t^2 \ t^3]$
- $\mathbf{P}$ = control points.
- $\mathbf{B}_{\text{natural}}$ = basis matrix for natural cubic spline
- Natural cubic splines support upto C2 continuity.
- Natural cubic spline does not support local controllability.
- It means if we change few points in a curve, then entire spline must be recomputed.
- This is inefficient

Hermite Cubic Spline

- Alleviates the Natural cubic spline problem.
- Developed by Charles Hermite
- It uses four control points p_0 , p_1 , p_2 , and p_3 .
- p_0 and p_2 are boundary points at time $t=0$ and $t=1$.
- p_1 and p_3 are the first derivative of the polynomial at time $t=0$ and $t=1$.

Hermite Cubic Spline



Use two interpolating conditions and
two derivative conditions per segment

Ensures continuity and first derivative
continuity between segments

Hermite Cubic Spline

$$f(t) = a + bt + ct^2 + dt^3$$

$$f'(t) = b + 2ct + 3dt^2$$

$$p_0 = f(t = 0) = a + b.0 + c.0 + d.0$$

$$p_1 = f'(t = 0) = 0 + b + 2c.0 + 3d.0$$

$$p_2 = f(t = 1) = a + b + c + d$$

$$p_3 = f'(t = 1) = 0 + b + 2c.1 + 3d.1$$

From above system of linear equations, we get

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 \end{bmatrix}, \text{ So, we have } C^{-1} = B_{hermite} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -3 & -2 & 3 & -1 \\ 2 & 1 & -2 & 1 \end{bmatrix}$$

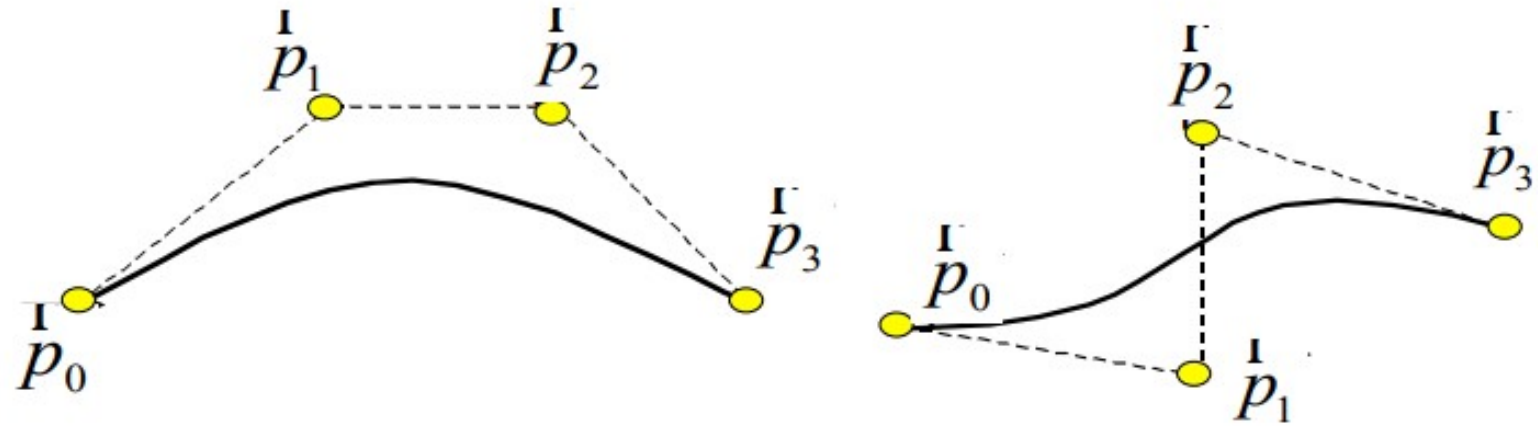
Hermite Cubic Spline

- It supports C1 continuity
- So, it is less smooth than natural cubic splines
- It supports local controllability
- It means we can modify some points of a curve without affecting the other curves of the spline.
- One problem is to specify the first derivative of both boundaries of each piece.

Cubic Bezier Spline

- Cubic Bezier Spline is widely used technique in computer graphics.
- It was introduced by French engineer to design Renault car bodies.
- It uses four control points p_0 , p_1 , p_2 , and p_3 .
- It is approximate cubic spline.
- Polynomial pieces do not pass all four control points.
- Instead, each piece originate at p_0 and ends at p_3 .
- Other two points determines convex hull.

Cubic Bezier Spline



Cubic Bezier Spline

- Each curve always lies within the convex hull of the control points.
- First derivative of end points can be computed from coordinate of control points.
- $f'(t=0)=3p_1-3p_0$ or $3p_1-3p_0$
- $f'(t=1)=3p_3-3p_2$ or $3p_3-3p_2$
- It does not support local controllability

Cubic Bezier Spline

$$f(t) = a + bt + ct^2 + dt^3$$

$$f'(t) = b + 2ct + 3dt^2$$

$$p_0 = f(t = 0) = a + b.0 + c.0 + d.0$$

$$f'(t = 0) = 3(p_1 - p_0) = 0 + b + 2c.0 + 3d.0$$

$$p_3 = f(t = 1) = a + b + c + d$$

$$f'(t = 1) = 3(p_3 - p_2) = 0 + b + 2c.1 + 3d.1$$

By rearranging we get,

$$p_0 = a$$

$$p_1 = a + b/3 + 0 + 0$$

$$p_2 = a + 2b/3 + c/3 + 0$$

$$p_3 = a + b + c + d$$

Cubic Bezier Spline

From above system of linear equations, we get

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1/3 & 0 & 0 \\ 1 & 2/3 & 1/3 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}, \text{ So, we have } C^{-1} = B_{\text{bezier}} \begin{bmatrix} 1 & 0 & 0 & 0 \\ -3 & 3 & 0 & 0 \\ 3 & -6 & 3 & 0 \\ 1 & 3 & -3 & 1 \end{bmatrix}$$

Cubic Bezier Spline

$$U = \begin{bmatrix} 1 & t & t^2 & t^3 \end{bmatrix} \text{ and, } P = \begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ p_3 \end{bmatrix}$$

$$f(t) = U \cdot B_{\text{bezier}} \cdot P$$

$$f(t) = \begin{bmatrix} 1 & t & t^2 & t^3 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ -3 & 3 & 0 & 0 \\ 3 & -6 & 3 & 0 \\ 1 & 3 & -3 & 1 \end{bmatrix} \cdot \begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ p_3 \end{bmatrix}$$

$$f(t) = \begin{bmatrix} 1 - 3t + 3t^2 + t^3 & 3t - 6t^2 + 3t^3 & 3t^2 - 3t^3 & t^3 \end{bmatrix} \cdot \begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ p_3 \end{bmatrix}$$

Cubic Bezier Spline

$$f(t) = p_0(1 - 3t + 3t^2 + t^3) + p_1(3t - 6t^2 + 3t^3) + p_2(3t^2 - 3t^3) + p_3.t^3$$

$$f(t) = p_0(1 - t)^3 + 3p_1t(1 - 2t + t^2) + 3p_2t^2(1 - t) + p_3.t^3$$

$$f(t) = p_0(1 - t)^3 + 3p_1t(1 - t)^2 + 3p_2t^2(1 - t) + p_3.t^3$$

Cubic Bezier Spline

- This shows that Bezier splines is weighted sum of some function of t .
- This function is called blending function.
- Blending function can represent a polynomial uniquely.
- The weights are parameter control points.

$$f(t) = \sum_{i=0}^n p_i BEZ_i^n(t)$$

,where $p_i(t)$ control points.

$$BEZ_i^n = \frac{n!}{(n-i)!i!} t^i (1-t)^{n-i}$$

Cubic Bezier Spline

- Construct Bezier curve described by control points (0,0), (1,2), (3,3), (4,0)

$$\text{We have, } C^{-1} \begin{bmatrix} 1 & 0 & 0 & 0 \\ -3 & 3 & 0 & 0 \\ -3 & -6 & 3 & 0 \\ 1 & 3 & -3 & 1 \end{bmatrix}, P^x = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 4 \end{bmatrix}, \text{ and } P^y = \begin{bmatrix} 0 \\ 2 \\ 3 \\ 0 \end{bmatrix}$$

At $t = 0$, We have
 $U = [1 \ 0 \ 0 \ 0]$

Now, from matrix multiplication,
 $f^x(t = 0) = x(t = 0) = U.C^{-1}.P^x$
we get, $x(t = 0) = 0$

$f^y(t = 0) = y(t = 0) = U.C^{-1}.P^y$
we get, $y(t = 0) = 0$

Cubic Bezier Spline

- Construct Bezier curve described by control points $(0,0)$, $(1,2)$, $(3,3)$, $(4,0)$

At $t = 0.2$, we have

$$U = [1 \quad 0.2 \quad 0.04 \quad 0.008]$$

Now, from matrix multiplication

$$f^x(t = 0.2) = x(t = 0.2) = U.C^{-1}.P^x$$

we get, $x(t = 0.2) = 0.704$

$$f^y(t = 0.2) = y(t = 0.2) = U.C^{-1}.P^y$$

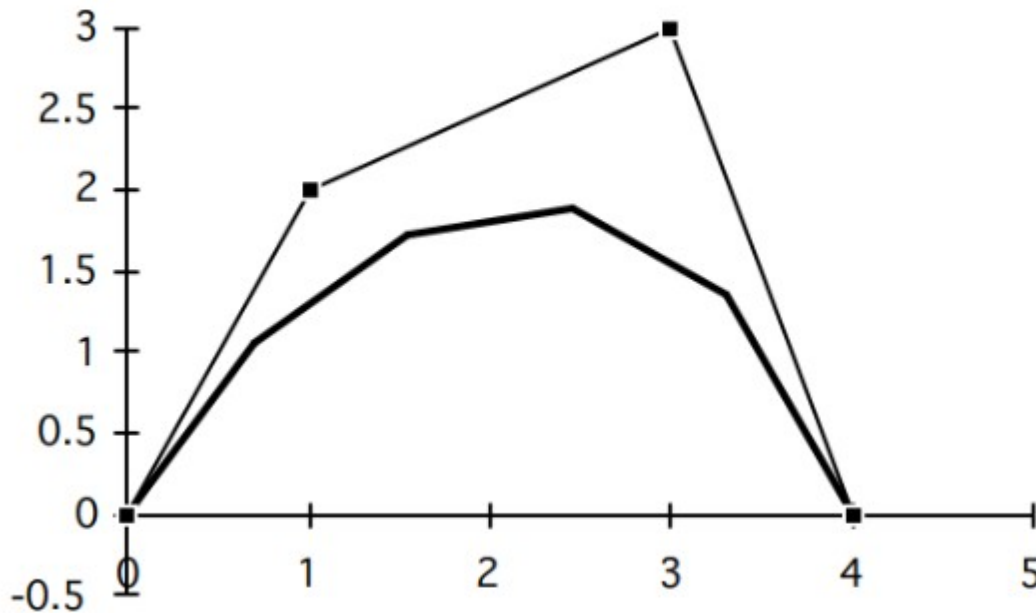
we get, $y(t = 0.2) = 1.056$

Cubic Bezier Spline

- Construct Bezier curve described by control points $(0,0)$, $(1,2)$, $(3,3)$, $(4,0)$
- $x(t=0)=0$ $y(t=0)=0$
- $x(t=0.2)=0.7$ $y(t=0.2)=1.1$
- $x(t=0.4)=1.55$ $y(t=0.4)=1.7$
- $x(t=0.6)=2.45$ $y(t=0.6)=1.9$
- $x(t=0.8)=3.3$ $y(t=0.8)=1.3$
- $x(t=1)=4$ $y(t=1)=0$

Cubic Bezier Spline

- Construct Bezier curve described by control points $(0,0)$, $(1,2)$, $(3,3)$, $(4,0)$



Cubic Bezier Spline

- **Disadvantages**
- In Bezier curve, change of one control points affects the entire curve.
- Degree of curve is fixed by number of control points.

Cubic Bezier Spline

- **Disadvantages**
- In Bezier curve, change of one control points affects the entire curve.
- Degree of curve is fixed by number of control points.

B-spline

- It is another approximating spline
- It supports both C2 continuity and local controllability
- B-splines are made up of made up of segments of some order k .
- So, we have spline made up of basis splines.
- If number of control points be $n+1$, then degree of polynomial is n
- Let the order be of segments be k
- k must be chosen such that $2 \leq k \leq n+1$

B-spline

- Number of segments is $n-k+1$
- Knot values $t_0, t_1, \dots, t_{n+k}$

B-spline curve equation

$$P(u) = \sum_{i=0}^n P_i N_{i,k}(u) \quad t_{k-1} \leq u \leq t_{n+1} \quad (a)$$

$$N_{i,k}(u) = \frac{(u - t_i)N_{i,k-1}(u)}{t_{i+k-1} - t_i} + \frac{(t_{i+k} - u)N_{i+1,k-1}(u)}{t_{i+k} - t_{i+1}} \quad \left(\frac{0}{0} = 0 \right) \quad (b)$$

$$N_{i,1}(u) = \begin{cases} 1 & t_i \leq u \leq t_{i+1} \\ 0 & \text{otherwise} \end{cases} \rightarrow \text{At any value of } u, \text{ there should be only one non-zero } N_{i,1}(u) \quad (c)$$

$N_{i,k}$: degree $(k-1)$ of u , k : order (independent of number of control points n)

t_i : knot values, boundary of non-zero range of each blending function

t_0 (for $i=0$) to t_{n+k} (for $i=n$) are needed ($n+k+1$ values)

B-spline curve - Knot

- Knot: $t_0, t_1, \dots, t_{n+k}$
 - parameter range is determined by knots
 - Periodic knots
$$t_i = i - k \quad 0 \leq i \leq n + k$$
 - Non-periodic knots

$$t_i = \begin{cases} 0 & 0 \leq i < k & \text{duplicates } k \text{ times} \\ i - k + 1 & k \leq i \leq n \\ n - k + 2 & n < i \leq n + k & \text{duplicates } k \text{ times} \end{cases} \quad (d)$$

Expansion of curve equation

- Ex)

$K=3$, $\mathbf{P}_0$, $\mathbf{P}_1$, $\mathbf{P}_2$ non-periodic uniform B-spline

$$t_0=0, \quad t_1=0, \quad t_2=0, \quad t_3=1, \quad t_4=1, \quad t_5=1$$

$$0 \leq u \leq 1$$

↑

(=t₂)

↑

(=t₃)

$$N_{0,1}(u) = \begin{cases} 1 & t_0 \leq u \leq t_1 \\ 0 & \text{otherwise} \end{cases} \quad (u = 0)$$

$$N_{1,1}(u) = \begin{cases} 1 & t_1 \leq u \leq t_2 \\ 0 & \text{otherwise} \end{cases} \quad (u = 0)$$

$$N_{2,1}(u) = \begin{cases} 1 & t_2 \leq u \leq t_3 \\ 0 & \text{otherwise} \end{cases} \quad (0 \leq u \leq 1)$$

$$N_{3,1}(u) = \begin{cases} 1 & t_3 \leq u \leq t_4 \\ 0 & \text{otherwise} \end{cases} \quad (u = 1)$$

$$N_{4,1}(u) = \begin{cases} 1 & t_4 \leq u \leq t_5 \\ 0 & \text{otherwise} \end{cases} \quad (u = 1)$$

$$N_{1,2}(u) = \frac{(u - t_1)N_{1,1}}{t_2 - t_1} + \frac{(t_3 - u)N_{2,1}}{t_3 - t_2} = \frac{(1 - u)N_{2,1}}{1} = (1 - u)$$

$$N_{2,2}(u) = \frac{(u - t_2)N_{2,1}}{t_3 - t_2} + \frac{(t_4 - u)N_{3,1}}{t_4 - t_3} = \frac{uN_{2,1}}{1} = u$$

$$N_{0,3}(u) = \frac{(u - t_0)N_{0,2}}{t_2 - t_0} + \frac{(t_3 - u)N_{1,2}}{t_3 - t_1} = \frac{(1 - u)N_{1,2}}{1} = (1 - u)^2$$

$$N_{1,3}(u) = \frac{(u - t_1)N_{1,2}}{t_3 - t_1} + \frac{(t_4 - u)N_{2,2}}{t_4 - t_2} = u(1 - u) + (1 - u)u = 2u(1 - u)$$

$$N_{2,3}(u) = \frac{(u - t_2)N_{2,2}}{t_4 - t_2} + \frac{(t_5 - u)N_{3,2}}{t_5 - t_3} = u^2$$

$$\therefore P(u) = (1 - u)^2 P_0 + 2u(1 - u)P_1 + u^2 P_2$$